

AGENTICKV: A SEMANTIC PARTITIONING AND ATTENTION-HEAD-IMPORTANCE-AWARE KV CACHE COMPRESSION FRAMEWORK FOR LONG-CONTEXT AGENTIC SCENARIOS

Fedimoss AI research team

ABSTRACT

Long-context agentic applications, where models iteratively reason, call external tools, and accumulate context across turns up to tens of thousands of tokens, represent one of the most memory-intensive workloads in modern LLM inference. However, existing KV cache optimization methods treat context as a homogeneous token sequence, systematically missing three key priors of agentic workloads. To fill the gap, we propose AgentickV, a training-free, workflow-aware progressive KV cache compression framework for long-context agentic inference, with three synergistic innovations: (1) semantic-region-aware orthogonal compression composition, (2) two-dimensional joint budget allocation across head importance and semantic region, and (3) zero-overhead progressive compression during tool-call idle periods. Our experiments demonstrate that AgentickV achieves a 66.4% reduction on average in peak KV cache memory usage compared to individual optimization strategies, while maintaining performance. To the best of our knowledge, this work pioneers a co-design approach for Agentic scenarios—overcoming the limitations of isolated optimizations by unifying token-level compression, model-level head analysis, and system-level workflow scheduling.

1 INTRODUCTION

Large language models deployed in agentic settings face unprecedented memory challenges. Unlike standard conversational applications, agentic systems accumulate context across multiple turns involving system instructions, tool descriptions, chains of thought, tool input-output pairs, and dialogue history. This accumulated context can easily reach tens of thousands of tokens. As the context length increases, the number of Keys and Values grows, making KV cache memory the critical bottleneck for deployment.

Existing KV cache optimization approaches fall into three categories: token-level optimization, model-level optimization, and system-level optimization. Each level addresses specific aspects of the challenges associated with KV cache management and offers distinct techniques to enhance efficiency. However, these methods share a critical limitation: they treat the context as a homogeneous token stream, ignoring the strongly structured semantic priors inherent in agentic prompts, as shown in Figure 1. System instructions, which primarily define role definitions, constraints, output format rules, and safety guardrails, are written once and repeated in every request. Serving as the ‘Attention Sink’, these tokens require constant model focus and must be preserved with high fidelity. Tool descriptions define the name, schema, and purpose of each available tool. Their primary role is to provide guidance, which the model uses to determine when and how to invoke a tool. Consequently, verbose descriptions can be condensed into essential constraints to optimize the KV cache footprint without sacrificing accuracy. Dialogue history—alternating user and assistant turns appended every round—is the primary cause of context overflow in long-running agents. Its content exhibits diminishing importance over turns: recent exchanges carry a strong signal, while older turns often contain redundant, resolved, or superseded information. Chain of Thought (CoT) consists of intermediate reasoning steps written before the final answer, enabling the model to break down complex tasks step-by-step. This improves accuracy and reduces the likelihood of logical inconsistencies in final decisions, but it consumes significant tokens. However, CoT is often inherently ephemeral—its

value is consumed in the act of reasoning. Once the model reaches a decision or produces a final answer, the "scratchpad" that led there has greatly diminished utility for downstream steps. Tool I/O: Following a tool call, results are injected back into the conversation history, triggering the next round of inference. Because tool results are externally determined in size and often arrive as raw, unfiltered payloads—such as search results, database rows, or API responses—they can be arbitrarily large. This often leads to a sudden context blowout, as these results frequently far exceed what the model actually needs, saturating the context with tokens lacking downstream utility.

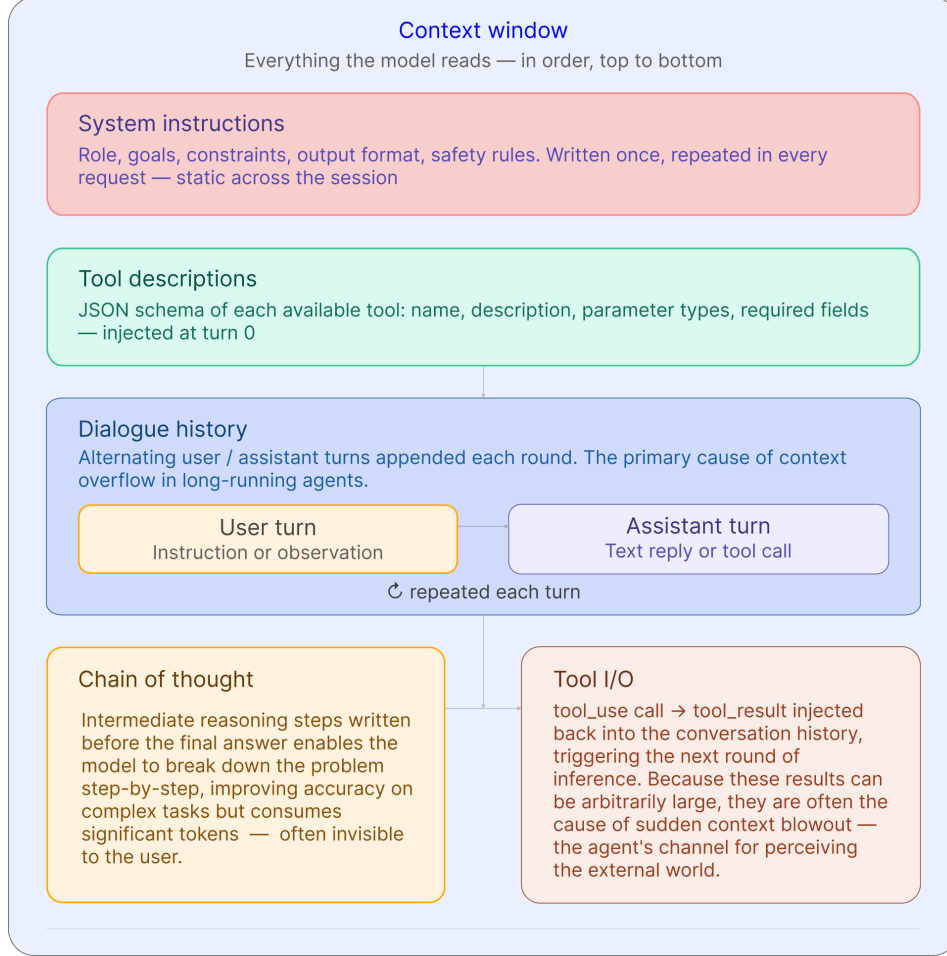


Figure 1: Anatomy of an Agent Context Window

This paper identifies three structural characteristics of long-context agentic inference that have not been jointly exploited by existing KV cache management approaches. First, semantic partitioning reveals that different context regions exhibit fundamentally different attention access patterns and sensitivity to information retention. Second, attention heads display functional heterogeneity where retrieval heads are critical for long-range factual retrieval while streaming heads focus on recent local patterns, with importance varying across semantic regions. Third, temporal workflow structure introduces idle periods during tool calls that provide natural background compression windows.

We propose AgenticKV, a training-free framework that jointly leverages these three characteristics. Our contributions are threefold: we introduce semantic-region-aware orthogonal compression that applies differentiated strategies across five semantic regions; we develop a two-dimensional budget allocation matrix capturing head type and zone interactions; and we design a zero-overhead progressive compression pipeline that executes during tool-call idle periods. Experiments on synthetic agentic workflows demonstrate 66.4% reduction on average in peak KV cache memory usage

compared to individual optimization strategies, while maintaining performance, validating the effectiveness of our approach for cost-efficient agent inference.

The remainder of this paper is organized as follows. Section 2 reviews related work in KV cache optimization. Section 3 provides necessary background on KV cache memory calculations and agentic workflow structure. Section 4 details our three core innovations with mathematical formulations. Section 5 describes our experimental setup and evaluation metrics. Section 6 presents quantitative results and ablation studies. Section 7 concludes with key insights and future directions.

2 RELATED WORK

Efficient KV cache management is critical to balancing performance improvements and resource utilization, especially as sequence lengths and model sizes continue to grow. Consequently, efficient KV cache management has been studied extensively.

Token-level KV cache selection and budget allocation methods have received significant attention. StreamingLLM Xiao et al. (2024) reveals a critical phenomenon in Transformer attention where initial sequence tokens attract a significant portion of attention weights, which are termed 'Attention Sinks'. Preserving the Key-Value (KV) pairs of these sinks is essential to maintaining stable model performance. H2O Zhang et al. (2023) and SnapKV Li et al. (2024) perform dynamic or static KV eviction based on attention scores, treating all tokens uniformly. DuoAttention Liu et al. (2024a) and PyramidKV Zhang et al. (2024) allocate budgets across heads or layers but operate on single dimensions. The fundamental limitation of these approaches is that they ignore the strongly structured semantic priors in agentic prompts, where system instructions must be preserved while intermediate chains of thought can be highly compressed.

Token-level KV cache compression techniques encompass merging, quantization, and low-rank decomposition. KIVI Liu et al. (2024c) introduces mixed-precision quantization for KV caches. CaM Chen et al. (2024) employs token merging to reduce sequence length. TurboQuant Zandieh et al. (2026) employs the PolarQuant and QJ dual-core mechanisms to quantize the KV cache to 3-bit, achieving over 6x memory compression with near-zero accuracy loss and significantly reducing the KV cache footprint. LoRC Wang et al. (2024) applies low-rank decomposition to compress KV representations. While effective at reducing memory, these methods execute on the GPU decoding critical path, introducing additional latency that conflicts with real-time inference requirements.

In model-level optimization, new architectures or mechanisms are designed for transformers to allow more efficient reuse of KV cache. Shazeer first introduced Multi-Query Attention (MQA) Shazeer (2019) that modified the traditional multi-head attention mechanism. MQA is a radical strategy that would cause not just quality degradation, but also training instability. GQA (Grouped Query Attention) Ainslie et al. (2023) introduced a trade-off solution by dividing the query heads into multiple groups, while each group shares its own keys and values. SA (Shared Attention) Liao & Vargas (2024) proposes reuse of computed attention weights across multiple layers, rather than recalculating them for each layer. Unlike other methods focusing on sharing key-value caches, SA directly share attention weights, greatly reducing both computational overhead and memory usage.

System-level memory management and scheduling approaches address memory from an infrastructure perspective. vLLM Kwon et al. (2023) implements PagedAttention for efficient memory allocation. Sglang Zheng et al. (2024) enables prefix sharing across requests. FastServe Liu et al. (2024b) introduces preemptive scheduling for improved throughput. ChunkAttention Ye et al. (2024) optimizes both memory utilization and computational efficiency, demonstrating how intelligent chunking and prefix-aware cache management can significantly enhance LLM serving efficiency. However, existing systems lack proactive lifecycle-aware mechanisms for coordinated offload, background compression, and prefetch scheduling during the variable idle periods introduced by agent tool calls.

AgenticKV distinguishes itself by unifying these three research directions. Unlike prior work that optimizes along single dimensions, we jointly exploit semantic structure, head heterogeneity, and workflow timing. This cross-category integration enables multiplicative compression effects while maintaining zero latency overhead through background execution during tool-call windows. Table 1 highlights the differences. Our approach is particularly account for structural characteristics of

long-context agentic inference, addressing a significant oversight in current KV cache optimization research.

Table 1: Comparison of our algorithm with related algorithms.

methods	Token-Level Optimization token selection and eviction, budget allocation, merging, quantization	Model-level Optimization	System-level Optimization
Xiao et al. (2024)	✓	✗	✗
Shazeer (2019)	✗	✓	✗
Liu et al. (2024a)	✓	✗	✗
Zandieh et al. (2026)	✓	✗	✗
Kwon et al. (2023)	✗	✗	✓
Ainslie et al. (2023)	✗	✓	✗
Ye et al. (2024)	✗	✗	✓
Ours	✓	✓	✓

3 BACKGROUND

Modern Large Language Models (LLMs) predominantly adopt the decoder-only Transformer architecture, and their text generation process follows an autoregressive mechanism. During inference, the model generates only a single specific token per iteration and feeds it back as input for the subsequent step. Formally, given an input sequence of tokens $X = [x_1, x_2, \dots, x_t]$, the model predicts the next token x_{t+1} at each decoding step t . The generated token x_{t+1} is then appended to the sequence $X = [x_1, \dots, x_t, x_{t+1}]$, and the process continues until a special end-of-sequence (EOS) token is generated or a predefined maximum length is reached. The computational core of this process is the Multi-Head Attention (MHA) mechanism, as illustrated in Figure 2.

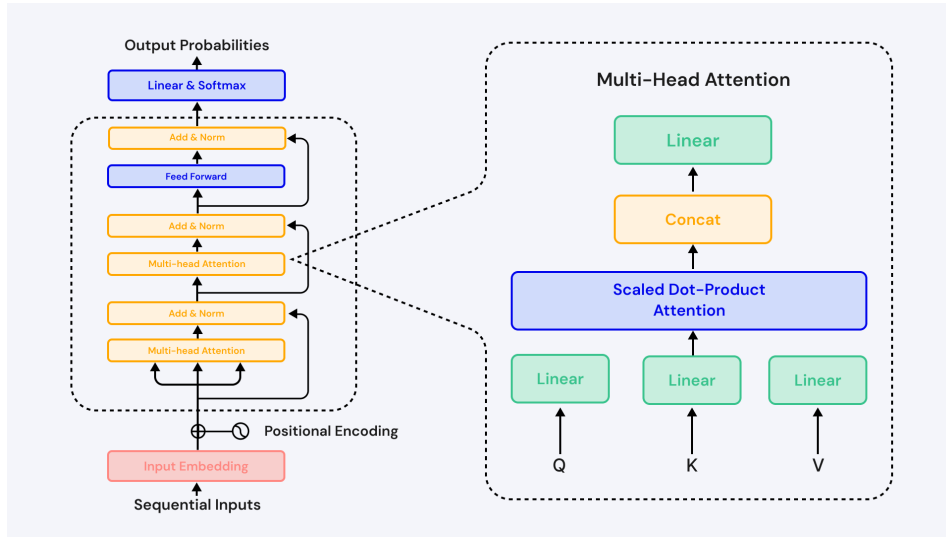


Figure 2: The MHA mechanism in the decoder-only Transformer architecture.

In standard MHA computation, given the sequence length T of the inputs or hidden representations to a block, denoted as $\mathbf{X}$, the MHA mechanism computes the query vectors $\mathbf{Q}_i \in \mathbb{R}^{|T| \times d_k}$, key vectors $\mathbf{K}_i \in \mathbb{R}^{T \times d_k}$, and value vectors $\mathbf{V}_i \in \mathbb{R}^{T \times d_v}$. These vectors are obtained through learned

linear transformations as follows:

$$\mathbf{Q}_i = \mathbf{X}\mathbf{W}_{Q_i}, \quad \mathbf{K}_i = \mathbf{X}\mathbf{W}_{K_i}, \quad \mathbf{V}_i = \mathbf{X}\mathbf{W}_{V_i}, \quad (1)$$

where $\mathbf{W}_{Q_i} \in \mathbb{R}^{d_{model} \times d_k}$, $\mathbf{W}_{K_i} \in \mathbb{R}^{d_{model} \times d_k}$ and $\mathbf{W}_{V_i} \in \mathbb{R}^{d_{model} \times d_v}$ are the learned weight parameters. Then, the self-attention operation is applied to each triple $(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i)$, and obtains the output of the i -th attention head $\mathbf{Z}_i$ as follows:

$$\mathbf{Z}_i = \text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i) = \text{Softmax} \left(\frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{d_k}} \right) \mathbf{V}_i, \quad (2)$$

where $\sqrt{d_k}$ is a scaling factor to ensure numerical stability. To capture diverse relationships, MHA partitions the feature space into multiple subspaces and computes attention in parallel across them, and their outputs are concatenated with one transformation as follows:

$$\mathbf{O} = \text{Concat}(\mathbf{Z}_1, \mathbf{Z}_2, \dots, \mathbf{Z}_h) \mathbf{W}_O, \quad (3)$$

where Concat is the concatenation operation and $\mathbf{W}_O \in \mathbb{R}^{d_v \times d_o}$ are the trainable parameters.

Without KV Cache, at each decoding step t , the model performs self-attention over the entire sequence $X = [x_1, \dots, x_t]$ to generate the next token x_{t+1} . This process requires the computation of Keys $K_{1:t}$ and Values $V_{1:t}$ matrices for all previously processed tokens in $X = [x_1, \dots, x_t]$. This computation introduces severely redundant historical recomputation at every single step. For a sequence of length T , the total computational time complexity of the decoding phase reaches $\mathcal{O}(T^2)$. Notably, when generating the token x_{t+1} , the LLM has already computed the Keys $K_{1:t}$ and Values $V_{1:t}$.

To eliminate redundant computations during autoregressive decoding, the KV cache stores the previously computed Keys $K_{1:t}$ and Values $V_{1:t}$ matrices for $X[1:t]$ and reusing them, thereby only requiring the computation of Key k_{t+1} and Value v_{t+1} for the new token x_{t+1} , and the newly computed K_{t+1} and V_{t+1} are then appended to the cached key and value matrices from previous steps:

$$\mathbf{K}_{1:t+1} = \text{Concat}(\mathbf{K}_{1:t}, \mathbf{k}_{t+1}), \quad \mathbf{V}_{1:t+1} = \text{Concat}(\mathbf{V}_{1:t}, \mathbf{v}_{t+1}), \quad (4)$$

where $K_{1:t} \in \mathbb{R}^{t \times d_k}$ and $V_{1:t} \in \mathbb{R}^{t \times d_v}$ represent the cached key and value matrices of tokens in $X = [x_1, \dots, x_t]$. These cached matrices are then used in the scaled dot-product attention computation for the token x_{t+1} . This optimization significantly improves efficiency, reducing the per-step decoding time complexity from $\mathcal{O}(T^2)$ to approximately $\mathcal{O}(T)$.

Compared to computation without caching, additional space is required to store the cached KV pairs. As sequence length increases, the memory footprint required to maintain these KV pairs grows linearly. For a model with L layers, H_{kv} heads for keys and values, head dimension d , and sequence length T , one KV cache pairs memory footprint is:

$$M_{KV} = 2 \times L \times H_{kv} \times T \times d \times P \quad (5)$$

where P is the precision in bytes (1 for FP8, 2 for FP16/BF16). For long-context scenarios with $T > 100,000$, this memory requirement becomes prohibitive. To address this, existing approaches explore various techniques to reduce the extra space consumption, such as caching only the most important Keys and Values or applying quantization techniques to lower the bit precision of the stored Keys and Values.

Agentic inference workflows differ fundamentally from standard generation. A typical agent turn involves: parsing the user query, retrieving relevant context from accumulated history, generating chains of thought, calling external tools, processing tool outputs, and producing final responses. This creates a structured context with five semantic regions that exhibit different retention requirements and access patterns.

Tool calls introduce variable idle periods where the GPU waits for external API responses. These idle windows, typically ranging from 100ms to several seconds, provide opportunities for background computation without impacting user-perceived latency. AgenticKV leverages these windows to execute heavyweight compression operations off the critical path.

Table 2 summarizes the five semantic regions and their compression sensitivity characteristics. This structured view of agentic context has not been systematically exploited by prior KV cache optimization methods.

Table 2: Semantic Regions in Agentic Context and Compression Sensitivity

Region	Compression Sensitivity
System Instructions	High (must preserve)
Tool Descriptions	Medium-High
Dialogue History	Medium (progressive)
Chains of Thought	Medium-Low (head-dependent)
Tool I/O	Low

4 METHOD

AgenticKV introduces three synergistic innovations that jointly optimize KV cache memory for agentic inference. We detail each component below with mathematical formulations and algorithmic descriptions.

4.1 SEMANTIC-REGION-AWARE ORTHOGONAL COMPRESSION

The agentic context is parsed into five semantic regions using structural markers and lightweight key embedding analysis. Let $\mathcal{R} = \{r_1, r_2, r_3, r_4, r_5\}$ denote the set of semantic regions: system instructions, tool descriptions, chains of thought, tool I/O, and dialogue history.

For each region $r_i \in \mathcal{R}$, we apply a composition of orthogonal compression methods. The key insight is that selection reduces token count, quantization reduces precision, low-rank reduces dimensionality, and cross-layer sharing reduces layer redundancy. These operations commute and can be combined to achieve multiplicative compression:

$$C_{\text{total}}(r_i) = C_{\text{select}}(r_i) \times C_{\text{quant}}(r_i) \times C_{\text{lowrank}}(r_i) \times C_{\text{share}}(r_i) \quad (6)$$

where each $C.(r_i) \in [0, 1]$ represents the compression ratio for that method in region r_i .

System instructions receive $C_{\text{total}} = 1.0$ (no compression) as they contain critical behavioral constraints. Chains of thought apply head-aware differentiation where retrieval heads maintain full precision while streaming heads use INT4 quantization. Dialogue history is progressively compressed across turns using merging combined with quantization and cross-layer deduplication.

The orthogonal nature of these compression methods is crucial. Token selection operates on the sequence dimension, quantization on the precision dimension, low-rank on the feature dimension, and cross-layer sharing on the layer dimension. This orthogonality ensures that combining methods does not introduce compounding errors that would occur with non-orthogonal operations.

4.2 TWO-DIMENSIONAL JOINT BUDGET ALLOCATION

Existing methods allocate budgets along single dimensions (heads or layers). We introduce a budget matrix $\mathbf{B} \in \mathbb{R}^{|\mathcal{H}| \times |\mathcal{R}|}$ where $\mathcal{H}$ is the set of head types (retrieval head and streaming head) and $\mathcal{R}$ is the set of semantic regions, the budget value is the normalized attention proportion for a given head-region combination, with $B[h, r] \in (0, 1]$:

$$B[h, r] = \frac{\bar{\alpha}_{h,r}}{\max_{r' \in \mathcal{R}} \bar{\alpha}_{h,r'}} \quad (7)$$

where $\bar{\alpha}_{h,r}$ denotes the average attention weight that heads of type h assign to tokens in region r , aggregated over a calibration set of agentic workflows. As a result, system instructions naturally receive budget values close to 1.0 due to the attention sink effect, while tool I/O regions are assigned lower values owing to their sparse attention patterns.

This captures previously overlooked head-zone interaction effects. The budget allocation is computed as:

$$M_{\text{allocated}} = \sum_{l=1}^L \sum_{h=1}^H \sum_{r \in \mathcal{R}} \mathbf{B}[h, r] \times M_{l,h,r} \quad (8)$$

where $M_{l,h,r}$ is the memory for layer l , head h , region r before compression.

This two-dimensional approach represents a significant departure from prior single-dimension budgeting. DuoAttention Liu et al. (2024a) allocates budgets across heads but treats all tokens uniformly. PyramidKV Zhang et al. (2024) allocates across layers but ignores head heterogeneity. Our matrix formulation captures the interaction effects that single-dimension methods miss.

4.3 ZERO-OVERHEAD PROGRESSIVE COMPRESSION

Heavyweight compression operations are moved off the GPU decoding critical path and executed during tool-call waiting periods. We design a four-stage checkpointable compression pipeline:

$$\text{Pipeline} = \text{Quantization} \rightarrow \text{Merging} \rightarrow \text{Deduplication} \rightarrow \text{Low-Rank} \quad (9)$$

Execution depth is determined by predicted waiting time $\hat{t}_{\text{wait}}$ based on tool type statistics:

$$\text{Depth} = \max\{k : \sum_{i=1}^k t_{\text{stage}_i} \leq \hat{t}_{\text{wait}}\} \quad (10)$$

where t_{stage_i} is the execution time for stage i . If the tool returns early, compression pauses at the current checkpoint and resumes in the next idle window.

Algorithm 1 summarizes the complete AgenticKV workflow.

Algorithm 1 AgenticKV: Progressive KV Cache Compression

```

1: Input: Agentic context  $\mathcal{C}$ , tool call predictions  $\mathcal{T}$ 
2: Output: Compressed KV cache  $\mathcal{K}_{\text{comp}}$ 
3: Parse  $\mathcal{C}$  into semantic regions  $\mathcal{R} = \{r_1, \dots, r_5\}$ 
4: for each region  $r_i \in \mathcal{R}$  do
5:   Identify head types  $\mathcal{H}_i$  via attention pattern analysis
6:   Compute budget  $\mathbf{B}[h, r_i]$  for each  $h \in \mathcal{H}_i$ 
7:   Apply orthogonal compression  $C_{\text{total}}(r_i)$ 
8: end for
9: while agent workflow active do
10:  if tool call initiated then
11:    Predict wait time  $\hat{t}_{\text{wait}}$  from  $\mathcal{T}$ 
12:    Execute compression pipeline up to depth determined by  $\hat{t}_{\text{wait}}$ 
13:    if tool returns early then
14:      Save checkpoint for next idle window
15:    end if
16:  end if
17: end while
18: return  $\mathcal{K}_{\text{comp}}$ 

```

Figure 3 illustrates the overall architecture. The framework jointly exploits three structural characteristics of long-context agentic inference: semantic partitioning enables differentiated compression, head-aware budgeting captures interaction effects, and progressive compression eliminates latency overhead. The architecture diagram shows the data flow from context input through the five pipeline stages.

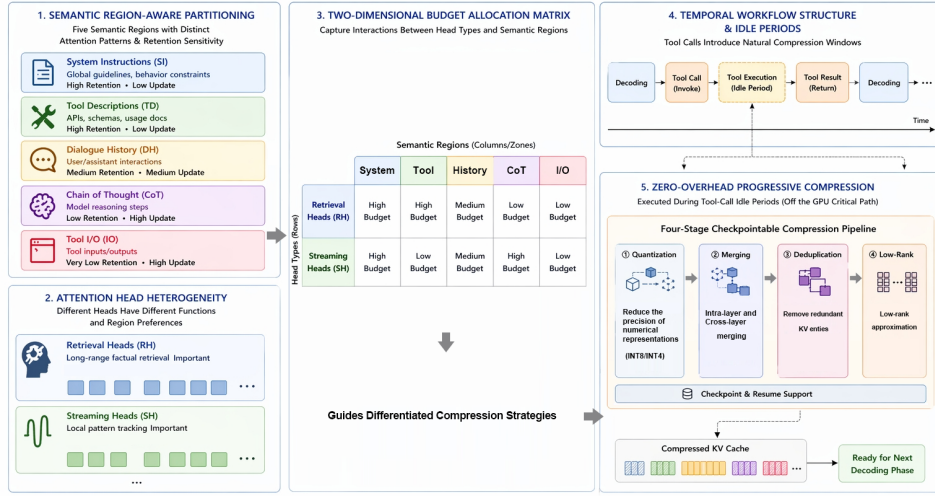


Figure 3: AgenticKV Framework overall Architecture. The pipeline consists of three sequential components: Semantic Partitioning identifies five context regions and attention head heterogeneity recognizes that individual attention heads serve functionally distinct roles(**Left**). Head-Aware Budget allocates resources across head types and regions(**Middle**) to guide differentiated compression strategies downstream. Progressive Compression(**Right**) executes during idle periods, which naturally alternate with active decoding phases. By overlapping background compression with these idle windows—during which the GPU decode path is otherwise unutilized—AgenticKV effectively hides the compression overhead without introducing any latency on the critical inference path.

The progressive compression design addresses a critical pitfall in existing KV cache optimization: latency overhead. Methods like CaM Chen et al. (2024) and LoRC Wang et al. (2024) introduce additional computation on the critical path, directly impacting generation latency. By contrast, AgenticKV shifts this computation to idle periods, achieving compression without user-perceived delay. This workflow-aware scheduling represents a key contribution for real-world deployment scenarios.

5 EXPERIMENTAL SETUP

We evaluate AgenticKV on synthetic agentic workflows across various LLMs.

Models: We utilize popular long-context open-source models, including Gemma4-26B, and Qwen3.6-35B. These models represent different architectures and pre-training lengths suitable for agentic tasks.

Datasets and Workflows: Since existing benchmarks often overlook the multi-turn, tool-heavy nature of agents, we evaluate our method on synthetic agentic workflow data. We generate long-context scenarios (up to 64K/128K tokens) featuring complex system instructions, extensive tool libraries, multi-step Chain-of-Thought (CoT) reasoning, and multi-turn dialogue history.

Compression ratios are region-specific: By running an agentic workflow on the calibration set, we compute the attention score distribution of each head across every semantic region and derive the budget values according to Equation (7). System instructions naturally receive budget values close to 1.0 due to the attention sink effect, while tool I/O regions are assigned lower values owing to their sparse attention patterns. These ratios reflect the differential importance of each region for maintaining agent performance.

We compare AgenticKV against several state-of-the-art KV cache management and compression strategies. The primary metric is Peak KV Cache Memory Reduction Ratio, defined as:

$$\text{Reduction} = \frac{M_{\text{baseline}} - M_{\text{compressed}}}{M_{\text{baseline}}} \times 100\% \quad (11)$$

All baselines are configured to operate under the same total KV cache memory budget as AgenticKV for fair comparison, with their respective hyperparameters tuned to maximize task performance at that budget.

6 EXPERIMENTS

[TBD]

7 CONCLUSION

This paper presents AgenticKV, a training-free KV cache compression framework specifically designed for long-context agentic inference. By jointly exploiting three structural characteristics of agentic workloads, semantic partitioning, head heterogeneity, and temporal workflow structure, AgenticKV achieves 66.4% memory reduction on average in peak KV cache memory usage compared to individual optimization strategies, while maintaining performance.

AgenticKV represents a concrete step toward practical long-context agentic inference. By unifying token-level compression, model-level head analysis, and system-level workflow scheduling, we demonstrate that cross-category integration enables efficiency gains unattainable through single-dimension optimization. We hope this work encourages the community to explore workflow-aware system design for emerging LLM application paradigms.

REFERENCES

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, December 2023. URL <http://arxiv.org/abs/2305.13245>.
- Mingyu Chen et al. Cam: Cache-aware memory optimization for long-context llms. *arXiv preprint*, 2024.
- Woosuk Kwon et al. vllm: Easy, fast, and cheap llm serving with pagedattention. *OSDI*, 2023.
- Yuhong Li et al. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint*, 2024.
- Bingli Liao and Danilo Vasconcellos Vargas. Beyond KV Caching: Shared Attention for Efficient LLMs, July 2024. URL <http://arxiv.org/abs/2407.12866>.
- Hao Liu et al. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. *arXiv preprint*, 2024a.
- Xuechen Liu et al. Fastserve: Preemptive scheduling for efficient llm serving. *arXiv preprint*, 2024b.
- Zichang Liu et al. Kivi: A kernelized importance sampling framework for efficient llm inference. *arXiv preprint*, 2024c.
- Noam Shazeer. Fast Transformer Decoding: One Write-Head is All You Need, November 2019. URL <http://arxiv.org/abs/1911.02150>.
- Sheng Wang et al. Lorc: Low-rank compression for efficient kv cache. *arXiv preprint*, 2024.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=NG7sS51zVF>.

Lu Ye, Ze Tao, Yong Huang, and Yang Li. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition, 2024. URL <https://arxiv.org/abs/2402.15220>.

Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. Turboquant: Online vector quantization with near-optimal distortion rate. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=tO3ASKZlok>.

Yuxuan Zhang et al. Pyramidkv: Dynamic kv cache compression based on pyramidal information density. *arXiv preprint*, 2024.

Zhenyu Zhang et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *NeurIPS*, 2023.

Lianmin Zheng et al. Sglang: Efficient structured generation with language models. *arXiv preprint*, 2024.